Intelligent Payment Fraud Investigation and Dispute Resolution Agent

Presentation + Complete Testing Guide

Participant: Radhika Mandan **Project:** `com.training:fraud-capstone` 1.0.0 **Stack:** Java 21 - Spring Boot 3.5.14 - Spring AI 1.1.8 - LangChain4j 1.22.0 - MySQL - Thymeleaf **Self-evaluation score:** 91/100 (Demo Ready) — see `SELF_EVALUATION_REPORT.md`

---

1. The Problem

When a customer disputes a card transaction, an analyst must pull evidence from many systems — transaction records, device data, location history, merchant risk, prior disputes, velocity, amount deviation, sanctions screening — then check bank policy and decide how to route the case.

This agent automates the **evidence gathering and analysis** — but a human always makes the final disposition. Recommend, never act.

2. Architecture

Pipeline: `Intake -> Plan -> Act -> Retrieve -> Assess -> Recommend -> Review -> Audit`

Stage	What happens	Code
Intake	Validates fields; rejects unmasked PANs, markup, non-synthetic data	`FraudCaseService.createCase()`
Plan	Picks the bounded tool plan	`InvestigationPlanner`
Act	Runs 8 allow-listed, read-only, timed, traced tools	`EvidenceTools` + `InvestigationOrchestrator`
Retrieve	Policy passages with doc/version/section/chunk/score; abstains below threshold; retired/expired docs excluded	`LexicalPolicyRetriever` + `PolicyIngestionService`
Assess + Recommend	Typed recommendation via the active processor	`FraudCaseProcessor.analyze()`
Validate	Schema + referential-integrity checks; forces `humanReviewRequired=true`	`RecommendationValidator`
Review	Human disposition with mandatory reason	`FraudCaseService.recordReview()`
Audit	Append-only hashed audit + tool traces	`AuditEventWriter`, `ToolTraceRecorder`

Three interchangeable "brains" — one contract `FraudCaseProcessor`

Mode	Class	How it works
`rules` (default)	`RulesFraudCaseAgent`	Deterministic rules R-001..R-008 over tool evidence; no API key needed
`spring-ai`	`SpringAiFraudCaseAgent`	`ChatClient` + typed `.entity()` + case-keyed `MessageWindowChatMemory`
`langchain4j`	`LangChain4jFraudCaseAgent`	`AiServices` + `@MemoryId` memory + `RetrievalAugmentor`

If a model fails or returns invalid/fabricated references, the validator rejects it and the case fails **safe** to `INCOMPLETE` for manual review.

Safety invariants (never broken)

- Human review is mandatory — the agent only recommends a route.
- No autonomous financial action (no blocks, refunds, reversals, customer contact).
- Tools are allow-listed, timed (2.5 s bound), traced with hashed args, and fail safely to structured results.
- Audit events and traces store hashes — never raw sensitive payloads.
- Memory is keyed by case ID — no cross-case leakage.
- Citations/evidence/trace IDs must exist in the case context — fabricated references are rejected, not persisted.

---

3. How to Run

Prerequisites

- Java 21, Maven, MySQL 8 running with DB `fraud_capstone`
- DB credentials via env (defaults `fraudapp/fraudapp123`): `DB_USER`, `DB_PASSWORD`
- For AI modes: `CLAUDE_API_KEY` (or `ANTHROPIC_API_KEY`), `LLM_BASE_URL`, `ANTHROPIC_MODEL`

Rules mode (default — no API key needed)

```
mvn spring-boot:run          # port 8090
```

Spring AI mode

```
CAPSTONE_PROCESSOR_MODE=spring-ai ANTHROPIC_MODEL=<model> SERVER_PORT=8
```

LangChain4j mode

```
CAPSTONE_PROCESSOR_MODE=langchain4j ANTHROPIC_MODEL=<model> SERVER_PORT
```

UI: `http://localhost:8090` — dashboard, intake form, case detail (evidence, traces, citations, chat, review).

---

4. How to Test the Whole Application

4.1 Automated suite (61 tests)

```
mvn clean test
```

Expected: **BUILD SUCCESS — 61 tests, 0 failures**. Coverage:

Test class	What it proves
`ApiIntegrationTest` (11)	Full API pipeline, dedup, unique IDs, PAN/markup rejection, memory isolation, review guards, state transitions
`RecommendationValidatorTest` (14)	Schema + referential-integrity: rejects fabricated evidence/trace/citation/rule IDs, version normalization, forced human review
`RulesFraudCaseAgentTest` (8)	R-001..R-008 rule outcomes incl. screening escalation and velocity/amount anomalies
`SpringAiAdapterContractTest` (4)	Stub `ChatModel`: valid output passes; fabricated IDs + malformed JSON fail safe
`LangChain4jAdapterContractTest` (7)	Stub service: same contract matrix + fence stripping
`LexicalPolicyRetrieverTest` (7)	Citation metadata, threshold abstention, top-K, retired-policy exclusion
`SyntheticEvidenceAdapterTest` (8)	Determinism + all scenario markers incl. VELOCITY/BIGAMT/SANCTION
`CaseStatusTransitionTest` (2)	Legal/illegal state transitions

4.2 Manual end-to-end script (rules mode)

```
# 1. Create a case (note the returned caseId — always max+1, never coll
curl -s -X POST localhost:8090/api/cases \
  -H 'Content-Type: application/json' -H 'X-Analyst-Id: analyst-1' -d '
   "reportedAt":"2026-11-10T10:15:00Z","channel":"WEB","maskedCustomerR
   "homeRegion":"EU-IE","transactionRef":"TXN-FRAUD-0042","amount":129.
   "occurredAt":"2026-11-10T09:40:00Z","transactionStatus":"POSTED",
   "transactionChannel":"CARD_NOT_PRESENT","maskedMerchant":"MRC-UNRECO
   "issueType":"UNRECOGNISED_TRANSACTION",
   "customerStatement":"I did not make this purchase.","cardPossession"
   "merchantRecognised":false,"syntheticDataConfirmed":true,"reviewRequ

# 2. Investigate (runs the whole pipeline)
curl -X POST localhost:8090/api/cases/<caseId>/investigations -H 'X-Ana

# 3. Inspect everything the agent produced
curl localhost:8090/api/cases/<caseId>/evidence        # typed, severit
curl localhost:8090/api/cases/<caseId>/tool-traces     # TR-xxx traces
curl localhost:8090/api/cases/<caseId>/recommendation  # validated sche
curl localhost:8090/api/cases/<caseId>/citations       # docId/version/
curl localhost:8090/api/cases/<caseId>/audit-events    # append-only au

# 4. Follow-up question (case-scoped, grounded)
curl -X POST localhost:8090/api/cases/<caseId>/messages \
```

```
  -H 'Content-Type: application/json' -H 'X-Analyst-Id: analyst-1' \
  -d '{"question":"What policy applies here?"}'

# 5. Human review — mandatory reason; ACCEPTED -> ROUTED, REJECTED -> C
curl -X POST localhost:8090/api/cases/<caseId>/reviews \
  -H 'Content-Type: application/json' -H 'X-Reviewer-Id: rev-1' \
  -d '{"disposition":"ACCEPTED","reason":"Evidence corroborates the cla
```

4.3 Scenario matrix — synthetic markers

Markers in references drive deterministic scenarios (see `SyntheticEvidenceAdapter`):

Marker (in ref)	Effect	Expected result
`TXN-FRAUD-*`	New device, distant location, auth unavailable	`UNAUTHORISED_TRANSACTION` -> `FRAUD_INVESTIGATION` (R-004)
`TXN-DUP-*`	Duplicate payment fingerprint	`DUPLICATE_PROCESSING` -> `PAYMENT_OPERATIONS` (R-001)
`TXN-GENUINE-*`	Clean trusted signals	`GENUINE_TRANSACTION` -> `MANUAL_REVIEW` (R-006)
`TXN-MISSING-*`	Transaction not found	`INSUFFICIENT_EVIDENCE` -> INCOMPLETE (R-002)
`MRC-BLOCK-*`	Merchant watch-list match	contributes to anomaly count
`CUST-*REPEAT*`	Multiple similar prior disputes	prior-dispute evidence
`CUST-TRUSTED/ LOCAL/DISTANT/ NEWDEV`	Device/location scenario control	adjusts anomalies
`VELOCITY` (cust or txn ref)	5+ txns in 10 min	VELOCITY/HIGH evidence item
`BIGAMT` (txn ref)	>5x baseline amount	amount-deviation/HIGH evidence
`SANCTION` (cust or merchant)	Screening match	R-008 -> forced `FRAUD_INVESTIGATION`

4.4 Behavioral test checklist

Test	How	Expected
Duplicate intake	POST the same intake body twice	Second call returns the **same caseId**; audit shows `DUPLICATE_INTAKE_RETURNED`
ID uniqueness after restart	Restart app, create case	New ID = max+1, no overwrite of existing rows
Long free text	>255-char statement, follow-up, review reason	All persist without errors (TEXT columns)
Retired policy	Ask about "legacy manual callback verification"	`POL-LEG-001` never appears; abstains or cites active docs only
Injection	`customerStatement` containing `<script>`	400 `validation_failed`

404 on static assets	App started before resources compiled	`mvn compile` then restart
Login denied DB	Wrong creds	Check `DB_USER`/`DB_PASSWORD` env vars